PROJECT REPORT

ShareNet

Campus Sharing Economy Platform

Submitted by: [HARISH S]

Register Number: [2116241001081]

Department: [INFORMATION TECHNOLOGY]

A mini-project report

Abstract

ShareNet is a student-focused campus sharing platform intended to make it easier for students to access, exchange and reuse useful items within a college community. Students may need textbooks, stationery, electronic devices, lab-related resources and other everyday items, while other students may have items they no longer use. These exchanges are often coordinated through informal groups and personal contacts, which can make discovery and communication difficult.

ShareNet proposes a centralized digital marketplace where students can list items for sale, offer items for rent or giveaway, publish requests through a wanted board, and report lost or found belongings. The platform also envisages user profiles, student-focused authentication and direct messaging to support communication and trust. The proposed technical architecture uses a React frontend, a Node.js and Express backend, and MongoDB for data storage, with supporting services such as JWT authentication, Socket.IO and Cloudinary. The report describes the problem, objectives, proposed solution, architecture, modules, requirements and possible future enhancements.

Keywords

Campus marketplace, resource sharing, student community, React, Node.js, Express, MongoDB.

Table of Contents

1. 1. Introduction
2. 2. Problem Statement
3. 3. Objectives
4. 4. Existing System and Its Limitations
5. 5. Proposed System
6. 6. System Requirements
7. 7. System Architecture
8. 8. Module Description
9. 9. Data Design
10. 10. Main Workflows
11. 11. Security and Privacy Considerations
12. 12. Testing Approach
13. 13. Advantages and Applications
14. 14. Limitations and Future Enhancements
15. 15. Conclusion
16. References

1. Introduction

A college community contains many resources that are used by students for a limited period. Examples include textbooks, calculators, electronics, project materials and other learning aids. At the same time, students may own useful items that are no longer required. A structured way to connect these students can reduce costs, improve access and encourage reuse.

ShareNet is proposed as a campus-oriented sharing economy platform. It brings several related activities—buying, selling, renting, giving away, lost-and-found reporting and item requests—into one digital environment. Rather than functioning as a general public marketplace, its central idea is to support exchanges within a student community.

2. Problem Statement

Students often find it difficult to discover affordable academic and everyday resources available within their own college community. Existing informal methods, such as messaging groups and personal networks, may scatter listings across multiple conversations and make it difficult to search, compare availability or contact the right person. A dedicated platform is needed to organize these exchanges and provide a consistent place for listings, requests and communication.

3. Objectives

- Create a centralized marketplace for student-to-student resource exchange.
- Support buying, selling, renting and giving away useful items.
- Provide a dedicated lost-and-found area and a wanted board.
- Enable students to discover listings and communicate with other users.
- Encourage reuse, affordability and more efficient use of available resources.
- Support a more trustworthy campus environment through user accounts and appropriate verification.

4. Existing System and Its Limitations

In an informal existing approach, students use general messaging groups, social media posts, notice boards or personal contacts to exchange items. These methods are familiar and easy to start using, but they are not designed specifically for structured campus transactions.

- Listings and requests can become buried in long message threads.
- There may be no consistent categories, filters or item details.
- Buyers and sellers must coordinate through separate communication channels.
- Lost-and-found information may be scattered across different groups.
- It can be difficult to identify reliable users or review previous interactions.

5. Proposed System

ShareNet proposes a single platform through which students can publish and browse item listings, request resources, offer items for free, and report lost or found belongings. User accounts and profiles provide an identity layer, while messaging supports discussion between interested students. The platform is intended to make campus exchanges more organized and discoverable.

5.1 Scope

The proposed scope covers student accounts, item discovery, listing management, rental and giveaway posts, wanted requests, lost-and-found posts and user-to-user communication. Physical handover, condition inspection and any payment arrangement may still need to be coordinated by the users unless a dedicated transaction or payment workflow is implemented.

6. System Requirements

6.1 Functional Requirements

- Users should be able to register, sign in and manage their profiles.
- Users should be able to create, view, update and remove their own listings, subject to permissions.
- Users should be able to browse and search available listings.
- The platform should support sale, rental and giveaway listing types.
- Users should be able to publish wanted posts and lost-and-found reports.
- Interested users should be able to communicate through the platform where messaging is enabled.
- The system should enforce access controls for protected actions.

6.2 Non-Functional Requirements

- Usability: interfaces should be clear and straightforward for students.
- Performance: common pages and listing searches should respond within a reasonable time.
- Security: passwords and protected routes must be handled using appropriate security practices.
- Maintainability: frontend, backend and data responsibilities should be separated.
- Reliability: input validation and error handling should reduce invalid or inconsistent data.
- Scalability: the architecture should allow additional users and modules to be introduced.

7. System Architecture

ShareNet is designed around a client-server architecture. The frontend presents the user interface and sends requests to the backend. The backend validates requests, applies application rules and uses database models to read or write information. Supporting services can provide real-time messaging and image hosting.

Layer	Technology	Responsibility
Frontend	React, HTML, CSS, JavaScript	User interface, navigation and client-side interactions
API communication	Axios, REST APIs	Sends requests and receives backend responses
Backend	Node.js, Express.js	Routes, validation, application logic and access control
Database	MongoDB, Mongoose	Stores users, listings, requests and related records
Authentication	JWT	Supports authenticated sessions and protected operations
Real-time communication	Socket.IO	Supports live messaging when implemented
Image storage	Cloudinary	Stores and serves listing images when configured

Typical request flow:

Student → React UI → Axios / REST API → Express Route → Middleware → Controller → Model → MongoDB

The backend response then travels back to the frontend, where the interface displays the result. Socket.IO may provide a separate real-time communication path, and Cloudinary may handle image files.

8. Module Description

8.1 User Registration and Authentication

Manages account creation, sign-in and access to protected features. Student verification may be used to help restrict participation to a campus community.

8.2 User Profile Management

Stores user-facing details and provides a place to view account information and reputation-related information, if enabled.

8.3 Product Listing and Discovery

Allows users to publish item details such as title, description, category, condition, price or rental terms, and images. Other users can browse and search listings.

8.4 Buying and Selling

Supports listings intended for sale and enables interested students to contact the listing owner to coordinate the exchange.

8.5 Rental Management

Provides a way to advertise items available for temporary use. Rental duration, availability and terms should be agreed by the users.

8.6 Giveaway Management

Allows users to list usable items that they are willing to give away without charge.

8.7 Lost and Found

Provides a dedicated area for posting information about missing or found belongings so that students can connect.

8.8 Wanted Board

Allows students to publish requests for specific items and lets other users respond if they can provide them.

8.9 Messaging

Supports communication between users about item details, availability and exchange arrangements. Real-time delivery can be provided using Socket.IO if configured.

9. Data Design

The following are conceptual data entities for the platform. The actual collection names and fields should be confirmed by inspecting the project's implemented database models.

Entity	Typical information
User	User ID, name, email, password hash, profile details, verification status
Listing	Listing ID, owner ID, title, description, category, listing type, price, condition, image references, status
Wanted Post	Post ID, creator ID, item or resource requested, description, date
Lost-and-Found Post	Post ID, creator ID, item description, lost/found status, location and date
Conversation / Message	Conversation ID, participant IDs, message content, timestamp and read status
Review / Reputation	Review ID, reviewer ID, reviewed user ID, rating and feedback

10. Main Workflows

10.1 Creating a Listing

17. The user signs in and opens the listing creation page.
18. The user enters item details and selects a listing type.
19. The frontend validates required fields and sends the data to the backend.
20. The backend checks authentication, validates the request and processes any image upload.
21. The listing data is stored in the database.
22. The backend returns a response and the frontend displays the new listing.

10.2 Contacting a Seller

23. A student browses or searches available listings.
24. The student opens a listing and reviews its details.
25. The student starts a conversation with the listing owner.
26. Both users discuss availability and exchange arrangements.
27. The users coordinate the physical handover and any agreed payment outside the platform unless an integrated workflow is available.

11. Security and Privacy Considerations

- Passwords should be stored using a secure password-hashing method, never as plain text.
- Protected API routes should verify authentication and authorization.
- The backend should validate and sanitize user input.
- Image uploads should be restricted by file type and size.
- Users should avoid publishing private contact details or sensitive personal information in public listings.
- Only the owner or an authorized administrator should be able to modify or remove protected records.
- Student verification should avoid exposing institutional credentials or unnecessary personal data.

12. Testing Approach

Testing should be carried out before the platform is presented as complete. The following test areas are recommended; this section is a test plan and does not claim that these tests have already passed.

- Authentication tests: valid and invalid sign-in, registration validation and protected-route access.

- Listing tests: create, view, update, delete, search and filter listings.
- Permission tests: confirm that users cannot modify another user's private listings.
- Messaging tests: verify conversation creation, message delivery and access restrictions.
- Upload tests: verify accepted image types, size limits and failed upload handling.
- Usability tests: check navigation, form clarity, responsiveness and error messages.

13. Advantages and Applications

- Helps students find resources at potentially lower cost.
- Makes existing campus resources easier to discover.
- Encourages reuse and can reduce avoidable waste.
- Combines several student exchange activities in one platform.
- Can support student communities, clubs, hostels and department-level sharing groups.
- Provides a practical example of a full-stack web application with multiple modules.

14. Limitations and Future Enhancements

The platform depends on students providing accurate listing information and coordinating exchanges responsibly. A digital listing alone cannot guarantee product condition, successful handover or user conduct. Any trust or verification mechanism should be supported by clear policies and appropriate moderation.

14.1 Future Enhancements

- Integrated payment support with secure transaction handling.
- QR-based handover or transaction confirmation.
- Improved search with location, category, price and availability filters.
- Mobile application support.
- Administrative moderation and reporting dashboard.
- Recommendation features based on user interests and listing activity.
- Analytics on popular categories and resource reuse.

15. Conclusion

ShareNet proposes a centralized campus sharing economy platform that connects students who have useful items with students who need them. By bringing together buying, selling, renting, giveaways, wanted posts, lost-and-found reporting and communication, the platform aims to make campus exchanges more organized and accessible. Its client-server architecture provides a foundation for a modular web application, while future improvements can strengthen trust, convenience and transaction support. ShareNet demonstrates how software can encourage collaboration, affordability and more responsible resource use within a student community.

References

1. ShareNet project repository: https://github.com/harish2802/share-net-mini-project

2. React documentation: https://react.dev/

3. Node.js documentation: https://nodejs.org/docs/

4. Express documentation: https://expressjs.com/

5. MongoDB documentation: https://www.mongodb.com/docs/

6. Socket.IO documentation: https://socket.io/docs/v4/

Preparation note: This is an editable draft report based on the ShareNet project description discussed. Before submission, replace the cover-page placeholders and verify the actual implemented features, folder structure, API routes, database schemas, screenshots and test results against your source code. Do not present proposed capabilities as completed functionality.